

Mean Stack Paper

Q1(a) What is the MEAN stack?

The **MEAN stack** is a collection of JavaScript-based technologies used to build full web applications — from the front end to the database.

It includes:

1. **MongoDB** – NoSQL database that stores data as documents
2. **Express.js** – Web framework for building backend APIs
3. **Angular** – Frontend framework for building dynamic UI
4. **Node.js** – Server environment that runs JavaScript

Example use:

Building a single-page application like a food-delivery app using JavaScript everywhere.

Q1(b) What is Node.js? Mention its advantages.

Node.js is a runtime environment that allows JavaScript to run on the server instead of only in the browser.

Advantages:

- **Very fast** because it uses the V8 engine
- **Non-blocking I/O**, meaning it can handle many users at once
- Uses **JavaScript on both front and back end**
- Has a huge library of modules through **npm**

Q1(c) What are AngularJS directives? Give types.

A **directive** in AngularJS is a special instruction added to HTML to give elements new behavior.

Example:

`ng-model="name"` binds an input box to a variable.

Types of directives:

- **Attribute directives** – modify element behavior (`ng-class`, `ng-model`)
- **Element directives** – custom HTML elements
- **Structural directives** – change the layout (`ng-repeat`, `ng-if`)
- **Component directives** – reusable UI blocks

Q1(d) What is middleware in ExpressJS? Why is it needed?

Middleware is a function that runs **between the request and the response** in an Express app.

Why needed?

- To check authentication
- To log requests

- To parse request body (JSON, form data)
- To handle errors
- To control access to routes

Example:

`app.use(express.json())` is middleware for reading JSON data.

Q1(e) Mention data types in MongoDB.

Common MongoDB data types include:

- **String**
- **Number (int, double, long)**
- **Boolean**
- **Date**
- **Object** (document)
- **Array**
- **ObjectId**
- **Null**
- **Binary Data**
- **Timestamp**
- **Regular Expression**

Q1(f) What are filters in AngularJS? List different filters.

Filters are used to **format data** before displaying it in the view.

Common filters:

- uppercase
- lowercase
- currency
- number
- date
- json
- orderBy
- filter (search)

Example:

`{{ price | currency }}` converts 1000 → ₹1,000.00.

Q1(g) What are the core modules in Node.js?

Core modules are built-in modules that come with Node, no installation needed.

Examples:

- **fs** (file system)
- **http**
- **https**
- **path**
- **os**
- **events**
- **stream**
- **crypto**
- **buffer**

Q2(a) – 3 Marks

Discuss the concept of asynchronous programming in NodeJS.

Node.js uses an **asynchronous (non-blocking)** programming model.

This means that Node.js does **not wait** for a task to finish.

Instead, it continues executing other tasks while the operation runs in the background.

Key Idea

In normal blocking code:

- One task must finish before the next starts.

In asynchronous code:

- Long operations like file reading, API calls, or database queries run in the background.
- Once they finish, a **callback**, **promise**, or **async/await** handles the result.

Why Node uses this model:

- Node.js runs on a **single thread**.
- To avoid blocking the thread, I/O operations are handled asynchronously.
- This allows Node.js to handle **thousands of concurrent users** efficiently.

Example:

```
console.log("Start");
```

```
setTimeout(() => {
```

```
console.log("This runs later");  
}, 2000);
```

```
console.log("End");
```

Output:

Start

End

This runs later

This shows that the timeout did not block the rest of the program.

Q2(b) – 3 Marks**Explain the term “Indexing” in MongoDB.**

An **index** in MongoDB is a special data structure that improves the **speed of searching** documents in a collection.

What indexing does:

- Works like an index in a book: instead of scanning all pages, MongoDB jumps directly to the matching records.
- Without an index → MongoDB performs a **collection scan** (slow).
- With an index → MongoDB uses **pointer-based structure** (fast).

Default Index:

MongoDB automatically creates an index on **_id** field.

Creating an Index:

```
db.students.createIndex({ name: 1 });
```

Example Scenario:

Searching for a student by name without index:

- MongoDB checks every document.

With index on "name":

- MongoDB jumps directly to matching entries → much faster.

Types of Indexes:

- Single field index
- Compound index (multiple fields)
- Text index
- Geospatial index

Q2(c) – 3 Marks

Discuss the life cycle of a component in AngularJS. Explain its different stages.

In AngularJS, a component (or directive) goes through a **life cycle** from creation to destruction. AngularJS provides functions to run code at each stage.

Main Life Cycle Stages:

1. Initialization Stage (Creation)

- Angular creates the component.
- `$onInit()` is called.
- Used to load data or initialize variables.

Example:

```
controller: function() {  
    this.$onInit = function() {  
        console.log("Component initialized");  
    }  
}
```

2. Digest / Change Detection Stage

- Angular checks for changes in variables using the **digest cycle**.
- `$doCheck()` is used to detect custom changes.

Example:

```
this.$doCheck = function() {  
    console.log("Checking for changes");  
}
```

3. Binding Update Stage (\$onChanges)

- Triggered when input bindings (`<component input="value">`) change.

4. Destruction Stage

- Called when component is removed from DOM.
- `$onDestroy()` cleans up timers, listeners, or intervals.

Example:

```
this.$onDestroy = function() {  
    console.log("Component removed");  
}
```

Summary:

1. `$onInit()` – when component is created
2. `$onChanges()` – when inputs change
3. `$doCheck()` – during every digest cycle
4. `$onDestroy()` – when component is destroyed

Q2(d) – 3 Marks

Difference between Route Parameters and Query Parameters in Express JS

1. Route Parameters

- Part of the **URL path**.
- Used to identify a specific resource.
- Defined using `:param`.

Example URL:

`/users/25`

Route:

```
app.get('/users/:id', (req, res) => {  
  res.send(req.params.id);  
});
```

Usage:

- Retrieving user by ID
- Fetching product details
- Dynamic routes

2. Query Parameters

- Appear **after ?** in URL.
- Used for filtering, sorting, searching, or pagination.

Example URL:

`/products?category=phones&sort=price`

Route:

```
app.get('/products', (req, res) => {  
  console.log(req.query.category);  
});
```

Key Differences Table

Feature	Route Params	Query Params
Location	Part of URL path	After ?
Purpose	Identify resource	Filter or sort data
Example	/user/5	/user?age=20
Access	req.params	req.query

Q2(e) – 3 Marks

Explain the advantages of using Node.js

Node.js is widely used because of the following benefits:

1. Fast Performance (V8 engine)

Node.js uses Google's **V8 JavaScript engine**, which compiles JavaScript into machine code and executes it very fast.

2. Asynchronous and Non-Blocking

- Multiple tasks can run in parallel.
- Perfect for apps that require fast I/O like chats, APIs, and streaming services.

3. Single Programming Language

Developers can use **JavaScript for both frontend and backend**, making development simpler.

4. Large NPM Ecosystem

- NPM provides **millions of packages**.
- Developers can quickly add functionality without writing from scratch.

5. Highly Scalable

Node can easily handle **thousands of concurrent users** using its event-driven model.

6. Real-time Applications Support

Ideal for:

- Chat applications
- Live notifications
- Online gaming
- Collaborative tools

Q3 a) Examine what is AngularJS? Provide a a brief overview of it with its main features. (4 Marks)

AngularJS is an **open-source JavaScript framework** developed by Google, primarily used for building **Single-Page Applications (SPAs)**. It is a **structural framework** that extends HTML's syntax to make it better suited for dynamic web applications.

Overview:

AngularJS follows the **Model-View-Controller (MVC)** or more accurately, the **Model-View-ViewModel (MVVM)** architectural pattern. Its core goal is to simplify the development and testing of SPAs by separating concerns: application logic (Model/Controller), presentation (View), and data synchronization.

Main Features:

1. **Two-Way Data Binding:** This is the most significant feature. It automatically synchronizes data between the **Model** (data in the `$scope`) and the **View** (HTML input elements) using the `ng-model` directive. Any change in the input field instantly updates the model, and vice-versa, minimizing manual DOM manipulation.
 - *Example:* `<input type="text" ng-model="user.name">`.
2. **Directives:** These are markers on DOM elements that tell the AngularJS HTML compiler to attach specific behavior or transform the DOM. They extend HTML capabilities.
 - *Examples:* **ng-app** (initializes the app), **ng-controller** (defines the scope), **ng-repeat** (loops over collections).
3. **Dependency Injection (DI):** AngularJS has a built-in DI subsystem that identifies a component's dependencies (e.g., Services, Controllers) and injects them. This makes components highly reusable, testable, and easier to manage.
4. **Scope (\$scope):** A plain JavaScript object that serves as the execution context for controllers and the communication bridge between the controller (Model logic) and the view (HTML). Scopes inherit hierarchically.

Q3 b) Demonstrate the Node.js event loop with an example. (4 Marks)

The **Node.js Event Loop** is the core mechanism that allows Node.js to perform non-blocking I/O operations despite being single-threaded. It continuously loops, checking the Event Queue for tasks (callbacks) that have completed their asynchronous operations (like file reads, database queries, network requests) and pushes them onto the Call Stack for execution.

Working Principle:

1. **Synchronous Execution:** All initial synchronous code runs first on the single main thread.
2. **Offloading:** When asynchronous I/O tasks are encountered, they are **offloaded** (e.g., to the system kernel or a worker pool via **libuv**).
3. **Loop Cycle:** The Event Loop enters a perpetual cycle, managing various queues in a specific order: **Timers, I/O Callbacks (Poll), Check (setImmediate), and Close Callbacks**.
4. **Microtasks:** High-priority queues like **process.nextTick()** and **Promise handlers** are always drained *before* the Event Loop moves to the next phase, ensuring their callbacks run with maximum urgency.

Example Demonstration:

JavaScript

```
console.log('1. Start Synchronous'); // 1. Executes immediately
```



```
// Goes to the Timer Queue

setTimeout(() => {
    console.log('4. setTimeout Callback (Timer Phase)');
}, 0);

// Goes to the Check Queue

setImmediate(() => {
    console.log('5. setImmediate Callback (Check Phase)');
});

// Goes to the Microtask Queue (Highest Priority)

process.nextTick(() => {
    console.log('3. process.nextTick Microtask');
});

console.log('2. End Synchronous'); // 2. Executes immediately
```

Execution Order and Rationale:

1. **Start Synchronous** (Synchronous code on the Call Stack).
2. **End Synchronous** (Synchronous code on the Call Stack finishes).
3. **process.nextTick Microtask** (Microtask queue is drained before the main loop starts).
4. **setTimeout Callback** (Event Loop enters the **Timer** phase).
5. **setImmediate Callback** (Event Loop eventually reaches the **Check** phase).

Q3 c) Demonstrate the significance of using ExpressJS in web development. (4 Marks)

Express.js is the minimal and flexible Node.js web application framework, serving as the **backend API layer** (the 'E' in MEAN). Its significance lies in providing structure and essential tools for building robust server-side applications and RESTful APIs.

Significance:

1. **Simplified API Development:** Express abstracts the complex, low-level details of Node.js's native HTTP module. It provides a clean, familiar, and easy-to-use set of tools (like `app.get()`, `app.post()`) for quickly defining API endpoints and handling requests.

2. **Robust Routing System:** It allows developers to map specific HTTP methods (GET, POST, PUT, DELETE) and URL paths to corresponding handler functions. This is crucial for creating well-organized, scalable, and **RESTful** APIs.
3. **Middleware Architecture:** Middleware is the core feature. These functions can execute tasks sequentially between receiving a request and sending a response. They are essential for handling cross-cutting concerns.
 - *Use Case Examples:* **Authentication/Authorization**, **Request body parsing** (e.g., JSON parsing), **Logging**, and **CORS configuration**.
4. **Performance and Scalability:** As it runs on Node.js, Express inherits its non-blocking I/O model, making it highly efficient at handling a large volume of concurrent connections, which is necessary for modern, high-traffic web applications.

Demonstration (Routing and Middleware):

JavaScript

```
const express = require('express');
```

```
const app = express();
```

```
// 1. Significance: Middleware (for JSON parsing and making data available)
```

```
app.use(express.json()); // Essential for handling incoming POST/PUT JSON data
```

```
// 2. Significance: Robust Routing (handling a POST request)
```

```
app.post('/api/products', (req, res) => {
```

```
  // req.body is available because of the express.json() middleware
```

```
  const newProduct = req.body;
```

```
  // Logic to save newProduct to MongoDB using Mongoose/Driver...
```

```
  console.log('Received Product:', newProduct.name);
```

```
  res.status(201).json({ message: 'Product created successfully', product: newProduct });
```

```
});
```

```
// 3. Significance: RESTful Endpoint (handling a GET request)
```

```
app.get('/api/products/:id', (req, res) => {
```

```
  const productId = req.params.id;
```

```
  // Logic to fetch product from DB using productId
```

```
  res.status(200).json({ id: productId, name: 'Sample Product' });
```

```
});
```

```
app.listen(3000, () => console.log('Server running on port 3000'));
```

Q3 d) Illustrate the concept of a Document in MongoDB. (4 Marks)

In **MongoDB**, the fundamental unit of data is a **Document**. MongoDB is a NoSQL, document-oriented database, and its structure is inherently flexible compared to a relational database's fixed table structure.

Concept of a Document:

1. **BSON Format:** A Document is a structure composed of **field and value pairs**, similar to JSON (JavaScript Object Notation), but stored internally in a binary representation called **BSON (Binary JSON)**. BSON allows for a richer set of data types than standard JSON (e.g., Dates, ObjectIDs, BinData).
2. **Dynamic Schema:** Documents within the same **Collection** (the equivalent of a table) do not need to have identical structures. Fields can be added or removed from documents without requiring a schema change for the entire collection. This flexibility speeds up development.
3. **Self-Contained and Denormalized:** Documents often store related data together using **embedded documents** and **arrays**. This denormalization means fewer database joins are required, leading to faster read performance.
4. **_id Field:** Every document requires a unique primary key field named **_id**. MongoDB automatically assigns an ObjectId if the field is not explicitly provided during insertion.

Example Illustration:

Consider a relational database where a user and their addresses are stored in two separate tables (User and Addresses), requiring a JOIN operation. In MongoDB, they can be stored in a single, embedded Document:

JSON

```
{
  "_id": ObjectId("6555c1d3..."),
  "firstName": "Alex",
  "lastName": "Johnson",
  "email": "alex.j@example.com",
  "status": "Active",
  "addresses": [ // Array of Embedded Documents
    {
      "type": "Shipping",
      "street": "123 Main St",
```

```

    "city": "Springfield",
    "zipCode": "12345"
  },
  {
    "type": "Billing",
    "street": "456 Corporate Blvd",
    "city": "Metropolis",
    "zipCode": "98765"
  }
],
"registrationDate": ISODate("2025-01-15T10:30:00Z")
}

```

This single Document is self-contained and holds all the necessary data for a user, including multiple nested address entries.

Q4 a) Explain the concept of form controls and form validation in AngularJS. Provide examples of different form controls and their usage. (4 Marks)

AngularJS simplifies working with forms by enhancing standard HTML forms and providing powerful built-in validation.

Form Controls and ngModel:

- **ngModel Directive:** This is the heart of form control in AngularJS. It is used to **bind** the value of an input element (<input>, <select>, <textarea>) to a property on the \$scope (the Model). It establishes the **two-way data binding**.
- **FormController:** When a <form> element is given a name attribute (e.g., <form name="myForm">), AngularJS attaches a FormController instance to the scope. This controller tracks the overall state of the form (e.g., \$valid, \$dirty) and manages all the contained ngModel controls.

Form Validation:

AngularJS validation automatically tracks the state of controls based on standard HTML5 attributes and AngularJS-specific directives.

1. **State Properties:** The ngModel controller exposes properties used for dynamic validation display:
 - **\$valid/\$invalid:** Overall validity of the control.
 - **\$pristine/\$dirty:** Whether the control has been untouched/modified.
 - **\$untouched/\$touched:** Whether the control has been visited (focus/blur).
2. **Validation Directives:** Directives are used to define validation rules:

- **required**: Checks for an empty value.
- **ng-minlength/ng-maxlength**: Checks string length.
- **type="email"**: Uses built-in pattern matching for email format.

Examples of Form Controls and Usage:

Control Type	HTML Example	AngularJS Purpose
Text Input	<code><input type="text" ng-model="user.name" required></code>	Binds to user.name and requires a value.
Checkbox	<code><input type="checkbox" ng-model="user.acceptTerms"></code>	Binds to a boolean user.acceptTerms.
Select Dropdown	<code><select ng-model="user.role" ng-options="r for r in roles"></code>	Binds the selected value to user.role from the roles array.

Validation Example:

HTML

```
<form name="myForm">

  <input type="email" name="emailField" ng-model="user.email" required>

  <div ng-show="myForm.emailField.$dirty && myForm.emailField.$invalid">

    <span ng-show="myForm.emailField.$error.required">Email is mandatory.</span>

    <span ng-show="myForm.emailField.$error.email">Invalid email format.</span>

  </div>

</form>
```

Q4 b) Explain the role of HTTP request and response in web development. (4 Marks)

The **HTTP (Hypertext Transfer Protocol)** request and response cycle is the fundamental communication mechanism of the web, operating on a **client-server model**. It dictates how clients (like web browsers or AngularJS apps) retrieve and interact with resources on a server (like a Node.js/Express app).

HTTP Request (Client $\rightarrow$ Server)

- **Role**: The client sends a message to the server, specifying the **action** it wants to perform and the **resource** it needs.
- **Key Components**:
 1. **Method (Verb)**: Defines the operation. Common methods are **GET** (retrieve data), **POST** (submit data), **PUT** (update data), and **DELETE** (remove data).
 2. **URI/Path**: Identifies the target resource (e.g., /api/products).

3. **Headers:** Contains metadata about the request (e.g., Host, User-Agent, Content-Type, Authorization token).
4. **Body (Optional):** Contains the data payload, typically used in POST/PUT requests (e.g., the JSON object of a new user).

HTTP Response (Server $\rightarrow$ Client)

- **Role:** The server processes the request and sends a message back to the client, indicating the **outcome** and providing the requested data (if successful).
- **Key Components:**
 1. **Status Code:** A critical three-digit code indicating the result (e.g., **200 OK**, **404 Not Found**, **500 Internal Server Error**, **201 Created**).
 2. **Headers:** Metadata about the response (e.g., Content-Type: application/json, Set-Cookie).
 3. **Body (Optional):** The actual content being returned. In MEAN stack APIs, this is usually **JSON data**.

Role in Web Development (MEAN Stack):

1. **Client-Server Interaction:** AngularJS uses the **\$http service** to make HTTP Requests (e.g., a GET request for a list of users).
2. **API Endpoints:** Express.js receives the request and, based on the URL and method, routes it to the correct handler.
3. **Data Exchange:** The Express handler interacts with MongoDB, fetches data, wraps it in an HTTP Response (Status 200, JSON body), and sends it back.
4. **UI Update:** AngularJS receives the Response, extracts the JSON data, and updates the **\$scope**, causing the view to refresh.

Q4 c) Describe features of Express JS. (4 Marks)

Express.js is a minimal, un-opinionated, and fast Node.js web application framework used to build robust server-side applications and APIs.

Key Features:

1. **Robust Routing System:**
 - Express offers a sophisticated mechanism to define routes based on URL paths and HTTP methods (e.g., `app.get('/', ...)`).
 - It supports **dynamic routing** using parameters (e.g., `/users/:id`) and allows for separate, modular route handlers (`express.Router()`), which is vital for building structured RESTful APIs.
2. **Middleware:**
 - This is the core concept where functions can access the request, response, and the `next()` middleware in the stack.

- Middleware is used to perform tasks on the fly, such as request body parsing (`express.json()`), session handling, logging, data validation, and authentication before the request hits the final route handler.

3. High Performance (Node.js Inheritance):

- Express leverages the asynchronous, non-blocking I/O architecture of Node.js. This design makes it highly efficient and scalable, particularly for I/O-heavy tasks like handling network traffic, making it excellent for serving APIs.

4. Minimalism and Flexibility:

- Express provides only the essential web application features, intentionally leaving out opinions on databases, templating engines, or complex ORMs.
- This minimalism ensures it is lightweight and gives the developer complete freedom to integrate specialized third-party tools (like Mongoose for MongoDB integration).

5. Integration with Templating Engines:

- While often used for APIs, Express supports the integration of various popular templating engines (like Pug or EJS) to generate dynamic HTML content on the server side when needed.

Q5 a) With reference to AngularJS, assess the following: (5 Marks)

This question requires an explanation of five core AngularJS concepts, with one mark generally allocated per concept.

1. Attribute Directive

- **Assessment:** An Attribute Directive modifies the **behavior** or **appearance** of an existing DOM element. It is used as an attribute on an HTML tag.
- **Example:** The **ng-class** directive dynamically adds or removes CSS classes to an element based on an expression.

HTML

```
<div ng-class="{ 'highlight': isSelected, 'error': hasError }">Content</div>
```

2. Structural Directive

- **Assessment:** A Structural Directive is responsible for changing the **DOM structure** by adding, removing, or manipulating elements. They are often prepended with an asterisk (*) in newer versions of Angular, but in **AngularJS**, they are direct directives like **ng-repeat**.
- **Example:** The **ng-if** directive removes an element and its contents from the DOM if the condition is false, and re-adds it if true.

HTML

```
<div ng-if="user.loggedIn">Welcome, {{user.name}}!</div>
```

3. Expressions

- **Assessment:** AngularJS Expressions are JavaScript-like code snippets written within **double curly braces** `{{ expression }}` in the HTML view. They are evaluated against the current \$scope and the result is rendered in the HTML.
- **Example:** They can perform basic arithmetic or string concatenation.

HTML

```
<p>Total: ${{ price * quantity }}</p>
```

4. Number Filter

- **Assessment:** A filter used to format a numeric expression for display, often used to specify decimal places or apply grouping separators (like commas for thousands). It is applied using the pipe (|) character.
- **Example:** To display a value with exactly two decimal places.

HTML

```
<p>Amount: {{ amount | number: 2 }}</p>
```

5. Date Filter

- **Assessment:** A filter used to convert a date object, timestamp, or date string into a readable, formatted date or time string. It accepts format strings to control the output.
- **Example:** To display the date in day/month/year format.

HTML

```
<p>Date: {{ eventDate | date:'dd-MM-yyyy' }}</p>
```

Q5 b) Review the Node JS process model. (5 Marks)

The Node.js Process Model is built on the philosophy of **single-threaded execution** combined with **non-blocking, asynchronous I/O** via the **Event Loop**. This architecture is what makes Node.js exceptionally fast and scalable for concurrent I/O operations (like handling web requests).

1. Single-Threaded Event Loop (V8 Engine):

- The main application code (JavaScript) runs on a single thread. This simplifies development by avoiding complex thread synchronization issues.
- The **Event Loop** is the mechanism that orchestrates execution. When the Call Stack is empty, the loop checks the Event Queue for completed asynchronous operations (callbacks) and pushes them onto the stack.

2. Non-Blocking I/O:

- When a request involves a long-running operation, especially **I/O-intensive tasks** (disk access, network requests, database queries), Node.js does **not** block the main thread.

- Instead, it immediately **offloads** the task to system resources or a pool of worker threads (managed by the **libuv** library). The main thread is then freed to serve other incoming requests.

3. Event-Driven Architecture:

- Node.js relies heavily on **events** and **callbacks**. When the offloaded I/O operation finishes, it triggers an event.
- The corresponding callback function is placed in the **Event Queue**. The Event Loop ensures the callback is eventually executed by the main thread.

4. Worker Pool (for heavy tasks):

- While the Event Loop handles most I/O efficiently, Node.js uses a **worker pool** (a small pool of operating system threads) for certain CPU-intensive tasks (like crypto hashing, file system operations) to ensure the single Event Loop thread remains free and responsive.

Summary: The model's strength is its ability to handle thousands of concurrent connections using minimal resources because it spends almost no time **waiting** for slow operations, instead utilizing the time to service other requests.

Q5 c) Justify why is error handling important in web applications? How does Express JS handle errors by default? (5 Marks)

Justification: Importance of Error Handling (3 Marks)

Error handling is essential for maintaining application **stability, security, and user experience (UX)**.

1. **Application Stability and Reliability:** Proper error handling prevents unhandled exceptions from crashing the server (in Node.js) or breaking the client-side logic (in AngularJS). It ensures the application remains running and resilient against unexpected conditions.
2. **Enhanced User Experience:** Instead of displaying cryptic errors, a blank page, or system stack traces, effective handling allows the application to show a meaningful, user-friendly error message (e.g., "Invalid input" or "Resource not found"). This guides the user and prevents frustration.
3. **Security and Information Hiding:** Default errors often reveal sensitive implementation details like database queries, server file paths, and technology stack versions. Custom error handlers suppress this information, replacing it with generic messages, which is a critical security measure against malicious attacks.
4. **Debugging and Maintenance:** A centralized error logging system ensures all necessary context (request data, stack trace, timestamps) is captured and logged internally, making it much easier for developers to trace, reproduce, and fix bugs quickly.

Express.js Default Error Handling (2 Marks)

By default, Express.js provides a simple, built-in error handler.

- When an error occurs synchronously within a route handler or middleware, or when an error object is explicitly passed to the `next()` function (`next(err)`), Express catches it.
- **Default Behavior:**

1. It automatically sets the HTTP response status code to **500 (Internal Server Error)**.
2. It sends a response to the client. In a **development environment**, this response often includes the full **stack trace**.
3. In a **production environment**, it sends a generic message to prevent security leaks.
4. It logs the error to stderr.

Overriding: This default behavior is almost always overridden by defining a **custom error-handling middleware** (one with four arguments: (err, req, res, next)) to ensure standardized, secure, and application-specific error responses (like JSON error bodies).

Q 6 a) Write the process of creating components in AngularJS application. Provide the command to create three components named home, about, dashboard. (6 Marks)

The request refers to creating components in AngularJS (v1.x). While modern Angular (v2+) uses the Angular CLI, AngularJS v1.x uses the **.component()** method on an AngularJS module. Since there is no command-line tool for generating AngularJS v1.x components, the answer must focus on the JavaScript definition and usage.

Process of Creating Components in AngularJS (v1.x)

Components in AngularJS are specialized, simplified directives that promote a component-based architecture, reusability, and isolation.

1. Module Setup:

Start by defining the main AngularJS module.

JavaScript

```
// app.module.js  
  
angular.module('myApp', []);
```

2. Component Definition:

Use the **.component()** method on the module. This method accepts the component's **name** (in camelCase, which will be used as the custom HTML tag in kebab-case) and a **Component Definition Object (CDO)**.

JavaScript

```
// home.component.js  
  
angular.module('myApp').component('homePage', {  
  
  // CDO Properties:  
  
  template: '<h1>Welcome Home!</h1><p>{{$ctrl.message}}</p>',  
  
  controller: function HomeComponent() {  
  
    // Component logic is isolated within this controller  
  
    this.message = 'This is the landing page.';  
  
  },  
});
```

```
// The default name for the controller instance in the template is $ctrl

controllerAs: '$ctrl',

// Always uses an isolated scope

bindings: {}

});
```

3. Component Integration (HTML View):

The component is used in the HTML template as a **custom element**, typically in kebab-case.

HTML

```
<body ng-app="myApp">

  <home-page></home-page>

</body>
```

Command to Create Components (Conceptual/Manual Definition)

Since AngularJS (v1.x) does not use a command-line interface (CLI) like modern Angular, there is no single ng generate component command. Instead, you manually create the JavaScript definition for each component.

The conceptual "command" or implementation involves defining three separate JavaScript component files:

Component Name	AngularJS Name (camelCase)	HTML Tag (kebab-case)	JavaScript Definition
Home	homeComponent	<home-component>	angular.module('myApp').component('homeComponent', { ... });
About	aboutComponent	<about-component>	angular.module('myApp').component('aboutComponent', { ... });
Dashboard	dashboardComponent	<dashboard-component>	angular.module('myApp').component('dashboardComponent', { ... });

Example of the dashboard Component Definition:

JavaScript

```
// dashboard.component.js

angular.module('myApp').component('dashboardComponent', {
```

```
// Component expects data from the parent ($)
bindings: {
  userData: '<'
},
template: `
  <div class="dashboard">
    <h2>Dashboard for {{$ctrl.userData.name}}</h2>
    <p>Welcome back! You have 5 new notifications.</p>
  </div>
`;
controller: function DashboardController() {
  // Initialization logic for the dashboard
}
});
```

Q 6 b) Write what is piping in Node JS streams? What is the chaining of streams in Node JS? How can piping and chaining be implemented with streams in Node JS? (6 Marks)

1. Piping in Node.js Streams (2 Marks)

- **Definition: Piping** is a mechanism used to connect the **output** of a **Readable Stream** to the **input** of a **Writable Stream**.
- **Purpose:** It is the standard and most efficient way to transfer data between streams in Node.js, automatically managing the flow of data.
- **Flow Control:** The pipe() method handles **backpressure**, ensuring a fast producer stream does not overwhelm a slower consumer stream by pausing the flow of data when the consumer cannot keep up.
- **Syntax:** readableStream.pipe(writableStream)

2. Chaining of Streams (2 Marks)

- **Definition:** Stream **chaining** is the act of linking multiple streams together sequentially using the pipe() method, where the output of one stream is piped directly into the input of the next.
- **Requirement:** This process typically involves one **Readable Stream**, one or more **Transform/Duplex Streams** (for data modification), and one **Writable Stream**.
- **Benefit:** It allows complex, multi-step data processing (like reading, compressing, and then writing a file) to be executed in a seamless, memory-efficient manner without loading the entire dataset into memory.

3. Implementation of Piping and Chaining (2 Marks)

The implementation relies solely on the pipe() method, which returns the destination stream, allowing for sequential calling.

Implementation Example (Chaining for Compression):

This example demonstrates reading a file (Readable), compressing it (Transform), and writing the result to a new file (Writable), all in a single chain.

JavaScript

```
const fs = require('fs');
```

```
const zlib = require('zlib'); // Provides the Transform Stream for compression
```

```
// 1. Readable Stream (Source)
```

```
const reader = fs.createReadStream('large_input.txt');
```

```
// 2. Transform Stream (Compressor)
```

```
const compressor = zlib.createGzip();
```

```
// 3. Writable Stream (Destination)
```

```
const writer = fs.createWriteStream('compressed_output.txt.gz');
```

```
// Implementing Chaining and Piping:
```

```
reader
```

```
  .pipe(compressor) // Output of reader (data) is input to compressor (transform)
```

```
  .pipe(writer)    // Output of compressor (compressed data) is input to writer (destination)
```

```
  .on('finish', () => {
```

```
    console.log('File successfully compressed via stream chaining.');
```

```
  });
```